INNOVATION & DESIGN THINKING (IDT) • PROJECT REVIEW 1 REPORT

# ApexLogic — Virtual Laboratory for Digital Electronics

Complete Technical Specification, Implementation Report & Viva Assessment Guide

| | | | |
|---|---|---|---|
| Project Title: | ApexLogic Virtual Laboratory | Target Domain: | Digital Electronics / System Design (CSE/ISE/ECE) |
| Live Deployment: | https://apexlogicde.netlify.app/ | Source Code: | github.com/Nilayansh/apexlogic |
| Current Status: | 100% Completed (Exceeds 50% Requirement) | Architecture: | Vanilla HTML5/CSS3/ES Modules • Zero Dependencies |

**Review 1 Executive Statement:** While the review guidelines require at least 50% project implementation and a working frontend demonstration, our team has achieved **100% project completion**. ApexLogic is fully built, thoroughly tested with automated assertions, and publicly deployed. This report provides everything our group needs to explain the problem, our architecture, the 5 experiments, and answer faculty questions.

## 1. Problem Identification (Why We Chose This Problem)

### 1.1 Background Context

In 1st and 2nd-year engineering programs across Computer Science (CSE), Information Science (ISE), and Electronics & Communication (ECE), **Digital Electronics / Logic Design** is a foundational laboratory course. Students learn how binary computation works by physically wiring discrete logic gates and 74-series Integrated Circuits (ICs) on solderless breadboards.

### 1.2 The Core Hardware Bottlenecks

Despite being central to computer architecture, traditional physical hardware laboratories face major friction points:

- **Component Vulnerability & Gate Burnout:** DIP IC chips (7400 NAND, 7408 AND, 7486 XOR) have fragile metal pins that bend and break upon breadboard insertion. Accidental short-circuits (+5V directly to Ground) destroy internal transistor gates. Students lose up to **70%–80% of their 3-hour lab session** tracking mundane continuity faults and replacing burnt chips rather than learning Boolean logic.
- **The "Black Box" Diagnostic Problem:** Commercial ICs are sealed in opaque black epoxy plastic. Students cannot observe intermediate internal nodes (such as signals $X_1$, $X_2$, $X_3$ inside an XOR gate or a Full Adder carry path). Multimeter probes slip and short adjacent pins, leaving students frustrated.
- **Instrument Scarcity:** Oscilloscopes and logic analyzers are expensive and scarce. In a 60-student batch lab, usually only 1 or 2 oscilloscopes exist. Consequently, sequential timing,

clock frequency division (f/2, f/4, f/8), and propagation delays remain purely abstract textbook equations.

- **Evaluation & Plagiarism Bottleneck:** Because breadboard assembly is slow and tedious, many students rush to copy truth table values from lab manuals without testing each binary input. Faculty cannot easily verify whether a student systematically verified all combinations.
- **Zero Access Outside College Hours:** Physical labs are locked outside scheduled hours. Students cannot practice or verify circuits before semester exams without purchasing costly hardware kits.

## 2. Proposed Solution (Our Concept & Impact)

**ApexLogic** is a browser-native, zero-dependency virtual laboratory designed with a clean **Swiss minimalist aesthetic** that delivers instant, mathematically authentic digital circuit simulation.

| TRADITIONAL LAB BOTTLENECK | APEXLOGIC PROPOSED SOLUTION | PEDAGOGICAL BENEFIT |
|---|---|---|
| Burned ICs, snapped pins, loose breadboard wires | **Mathematical Netlist Simulation:** Pure algorithmic execution where components never wear out or short. | Zero-risk, repeatable experimentation with instant reset. |
| Sealed "Black Box" ICs with hidden intermediate nodes | **Interactive Floating Logic Probe:** Allows students to tap any node to inspect Node Name, Voltage (+5V / 0V), and Logic State (1 / 0). | De-blackboxes ICs; students understand intermediate logic flow. |
| Scarcity of physical oscilloscopes | **Integrated 32-Sample Logic Analyzer:** Real-time SVG waveform strip graphing clock and divided frequencies. | Visual proof of frequency division (f/2, f/4, f/8) in sequential circuits. |
| Copied lab records & unverified truth tables | **Active Truth Table Tracking & Verified Stamp:** System automatically monitors visited rows; awards `VERIFIED ✓` only when 100% of states are tested. | Guarantees genuine student lab work; faculty can verify completion instantly. |
| Locked lab rooms & expensive kits | **100% Web-Native & Free:** Runs on any phone, tablet, or PC browser with zero installs or logins. | 24/7 self-paced practice before viva and practical exams. |

## 3. Frontend Development & User Experience

### 3.1 Design Philosophy: Swiss Functional Minimalism

Instead of cluttered cartoon diagrams or skeuomorphic breadboard graphics that distract students, ApexLogic follows clean **Swiss graphic design** (inspired by Dieter Rams):

- **Warm Canvas & Hairline Geometry:** Off-white background ( `#FBFBFB` ) with 1px borders ( `#E4E4E7` ) and 12px card radii.
- **Signal Color Hierarchy:** Logic HIGH (+5V) glows in crisp **Cobalt Blue ( `#1D4ED8` )** at 2px stroke width; Logic LOW (0V) rests in subdued neutral gray ( `#D4D4D8` ) at 1.5px; Clock signals illuminate in **Amber ( `#D97706` )**.
- **Tactile Physical Feedback:** Debossed toggle switch pill tracks with spring recoil physics (120ms cubic-bezier) and an optional 8ms synthetic click tone via the **Web Audio API**.

### 3.2 Navigation Architecture & Core Screens

1. **Home Screen ( `index.html` ):** Displays the 5 lab experiments in clean hairline rows with real-time progress indicators (e.g., `Rows 4/4 · Viva 3/3` ) stored in browser memory.
2. **Experiment Studio ( `experiment.html?id=XX` ):**
   - **Simulate Tab:** A 2-column layout pairing the interactive SVG schematic, tactile switches, and probe (left) with the live truth table, status readout, and waveform analyzer (right).
   - **Manual Tab:** Textbook-accurate lab documentation including Aim, Theory, Equations, IC pinouts, and step-by-step Procedure.
   - **Viva Tab:** 3 interactive conceptual questions with immediate answer lock, score tracking, and detailed technical rationale.
3. **Printable Lab Record ( `record.html?id=XX` ):** Centered A4-proportioned sheet with student USN, vector circuit snapshot, verification stamp, viva scorecard, and faculty signature line.

## 4. Implementation Status: All 5 Experiments (100% Complete)

**Experiment 01 — Universal Gate Realization (NAND-Only Logic)**

**Curriculum Aim:** Realize NOT, AND, OR, and XOR gates using exclusively IC 7400 Quad 2-Input NAND gates.

- **NOT Gate:** $Y = \overline{A \cdot A} = \overline{A}$ (1 NAND gate)
- **AND Gate:** $Y = \overline{\overline{A \cdot B}} = A \cdot B$ (2 NAND gates)
- **OR Gate:** $Y = \overline{\overline{A} \cdot \overline{B}} = A + B$ (3 NAND gates via De Morgan's Theorem)
- **XOR Gate:** $Y = A \oplus B = \overline{\overline{A \cdot X_1} \cdot \overline{B \cdot X_1}}$ where $X_1 = \overline{A \cdot B}$ (4 NAND gates)
- **Interactive Feature:** Dynamic badge overlays display live intermediate values ($X_1, X_2, X_3$). Truth table includes a live *Native Gate Comparator Column* proving the NAND network matches textbook logic.

**Experiment 02 — Half Adder & Full Adder**

**Curriculum Aim:** Construct binary arithmetic addition circuits with carry propagation.

- **Half Adder:** $\text{Sum} = A \oplus B$, $\text{Carry} = A \cdot B$ using IC 7486 (XOR) and IC 7408 (AND).
- **Full Adder:** $\text{Sum} = A \oplus B \oplus C_{\text{in}}$, $C_{\text{out}} = A \cdot B + C_{\text{in}} \cdot (A \oplus B)$ using 5 cascaded gates.
- **Interactive Feature:** Live decimal arithmetic line translates binary signals directly into human math: e.g., $1_2 + 1_2 + 1_2 = 3_{10} \implies \text{Sum} = 1, C_{\text{out}} = 1$.

**Experiment 03 — 4-to-1 Multiplexer (Data Routing)**

**Curriculum Aim:** Select and route 1 of 4 data inputs ($D_0$–$D_3$) using select lines ($S_1, S_0$) and active-low enable ($\bar{E}$).

- **IC Emulated:** IC 74153 Dual 4:1 Data Selector.
- **Industrial Convention:** Active-low enable ($\bar{E}$) forces output $Y = 0$ when $\bar{E} = 1$.
- **Interactive Feature:** Dynamic channel dimming — the selected data wire illuminates brightly in Cobalt Blue while unselected channels fade to 45% opacity. The active product term in the Boolean equation turns bold in real time.

**Experiment 04 — BCD to 7-Segment Decoder**

**Curriculum Aim:** Translate 4-bit Binary Coded Decimal ($0000_2$ to $1001_2$) into seven display signals ($a$ through $g$).

- **IC Emulated:** IC 7447 active BCD decoder driving a scalable SVG 7-segment module.
- **Bidirectional Synchronization:** An interactive decimal stepper ( `[-] 0 [+]` ) stays in sync with 4 binary switches ($B_3, B_2, B_1, B_0$).
- **Invalid Code Detection:** Feeding numbers $10$–$15$ lights only the center dash (segment $g$) and displays an alert badge: `Invalid BCD Input (Range: 0-9)`.

**Experiment 05 — 3-bit Asynchronous Ripple Counter**

**Curriculum Aim:** Observe binary up-counting ($000_2 \to 111_2$) and sequential clock frequency division ($f/2, f/4, f/8$).

- **Architecture:** 3 cascaded positive-edge triggered T flip-flops ($T = 1$) clocked by $\overline{Q_n}$ (which rises on $Q_n$'s falling edge, generating an up-count).
- **Interactive Feature:** Manual single-step clock button and variable-speed auto-oscillator ($0.5\text{ Hz}$ to $4.0\text{ Hz}$).
- **Logic Analyzer:** Real-time 32-sample scrolling SVG waveform graphing synchronized channels for $\text{CLK}$, $Q_0$, $Q_1$, and $Q_2$.

## 5. Under-the-Hood Engineering (What We Used & Why)

### 5.1 The Pure Web Architecture (No React, No npm, No Build Tools)

We deliberately rejected heavy frameworks (React/Vue/Angular) and NPM packages. Why?

- **Zero Toolchain Friction:** No Webpack/Vite bundlers that fail or require Node.js installs on campus computers.
- **Sub-Millisecond Loading:** The entire application payload is under 80 KB uncompressed. It loads instantly on mobile 3G/4G networks.
- **Long-Term Archival Stability:** Plain HTML5, CSS3, and native JavaScript ES modules will continue to run without breaking 10 years from now.
- **Client-Side Privacy:** Student USN, completed lab rows, and quiz scores reside securely in the browser's `localStorage` (with in-memory fallback), requiring no database or user tracking.

### 5.2 The Topological Netlist Evaluator ( `engine.js` )

ApexLogic is **not** a pre-recorded animation. It is a genuine netlist evaluation engine:

- Circuit components are declared in topological order (a component only reads nets produced by previous gates or inputs).
- When an input switch toggles, `evaluateNetlist(circuit, inputs)` recalculates the entire netlist graph in microseconds:

```
export function evaluateNetlist(netlist, inputs) {
  const nets = { ...inputs };
  for (const c of netlist.components) {
    const ins = c.in.map(n ⇒ nets[n]);
    if (c.type ≡ 'BLOCK') Object.assign(nets, c.fn(ins));
    else nets[c.out] = GATES[c.type](...ins);
  }
  return nets;
}
```

### 5.3 The Orthogonal Wire Router ( `renderer.js` )

To produce textbook-quality schematics automatically, we engineered a deterministic routing algorithm using pure SVG elements:

- Traces are routed using clean orthogonal lines: straight horizontal segments if $sy == dy$, or dedicated routing channels ($cx = dx - 24 - 8k$) left of destination columns if $sy \neq dy$.
- Junction dots ($r = 3\text{px}$) are placed automatically on fan-out driver pins and channel bend points ($n > 1$).
- Every wire carries a parallel transparent 14px hit-box path for responsive logic probe touch/hover hit-testing.

## 6. Academic Evaluation: The Printable Lab Record

To eliminate the practice of blindly transcribing lab records, ApexLogic features a dedicated printable lab record module ( `record.html` ):

- **Tamper-Evident Verification:** If a student tests every required truth table row, the sheet receives a bordered green `VERIFIED ✓` stamp. If any row was skipped, it displays an amber warning: `INCOMPLETE — n/N rows tested`.
- **Circuit State Snapshot:** Renders a high-resolution monochrome vector SVG reflecting the exact switch state the student tested.
- **Viva Voce Scorecard:** Summarizes the student's quiz score (e.g., $3/3$) alongside a breakdown of their chosen answers.
- **Single-Page Guarantee:** Governed by CSS `@page { size: A4; margin: 12mm; }` and strict print media queries, ensuring the Aim, Theory, Diagram, Table, Viva Score, and Faculty Signature fit onto **exactly one A4 sheet**.

## 7. Individual Viva Preparation: Questions Faculty Will Ask & How to Answer

Since marks are awarded on **Individual Performance**, every group member must master these answers:

**Q1: "Is this just an interactive animation, or are you actually simulating the circuit logic?"**

**Answer:** "It is a true simulation engine! In `engine.js`, we evaluate the circuit netlist graph topologically. When an input switch is toggled, logic states propagate through gate primitives (NAND, AND, OR, XOR) or IC block functions on the fly. The truth table cursor, voltage probe, and output lamps reflect real computed values, not hardcoded animations."

**Q2: "What is the Logic Probe tool, and why did you build it?"**

**Answer:** "In physical hardware labs, chips are sealed black boxes—students cannot observe intermediate gates. Our Floating Logic Probe turns the cursor into an electronic test probe. Hovering over or tapping any wire or pin immediately inspects its Node ID, voltage (+5.0V for HIGH, 0.0V for LOW), and binary logic state (1 or 0)."

**Q3: "Why did you build this in Vanilla JavaScript instead of React, Next.js, or Flutter?"**

**Answer:** "For three reasons: First, **zero build bloat**—the entire application loads in under 100ms on low-spec lab desktops. Second, **long-term stability**—there are no npm dependencies that break over time. Third, **zero hosting overhead**—it is 100% client-side and can be deployed anywhere without cloud servers or databases."

**Q4: "How does the ripple counter work, and why does it count up?"**

**Answer:** "We use positive-edge triggered T flip-flops with $T=1$ (toggling every clock). To count up, stage $n+1$ is clocked by $\overline{Q_n}$. When $Q_n$ falls ($1 \to 0$), its complement $\overline{Q_n}$ rises ($0 \to 1$). That rising edge triggers the next stage, producing the natural binary up-count sequence $000_2 \to 111_2$."

**Q5: "How does your system prevent students from cheating on their lab records?"**

**Answer:** "ApexLogic tracks every truth table row in `localStorage`. The Lab Record only receives a green `VERIFIED ✓` stamp if the student has actively tested 100% of the required combinations. Otherwise, it stamps `INCOMPLETE — n/N rows tested`. It also prints the student's USN and a snapshot of their exact tested circuit."

**Q6: "How did you verify that your simulation has no bugs?"**

**Answer:** "We built an automated test suite at `/dev/selftest.html`. It runs programmatic unit assertions verifying all 128 multiplexer combinations, all adder states, all 16 BCD decoder outputs, and an 8-cycle sequence check on the counter. All assertions pass with 100% success."

---